

CineSeat Project: Final Retrospective & Future Roadmap

Project: 6-Week Junior
Developer Internship |
September 2026

Table of Contents

1. Executive Summary
2. Project Retrospective
 - 2.1 Technical Retrospective
 - 2.2 Process Retrospective
 - 2.3 Collaboration & Knowledge Transfer
3. Evaluation Metrics & Key Performance Indicators
 - 3.1 Code Quality Metrics
 - 3.2 System Performance KPIs
 - 3.3 Timeline & Delivery Metrics
4. Lessons Learned by Phase
 - 4.1 Planning Phase (Week 1)
 - 4.2 Architecture Phase (Week 2)
 - 4.3 Implementation Phase (Week 3)
 - 4.4 Testing Phase (Week 4)
 - 4.5 Code Review & Finalization (Week 5)
5. Risk & Mitigation Analysis
 - 5.1 Technical Risk Assessment
 - 5.2 Organizational Risk
6. Future Roadmap (90-Day Sprint Plan)
 - 6.1 Roadmap Philosophy
 - 6.2 Sprint 1: Foundation (Days 1–30)
 - 6.3 Sprint 2: Resilience (Days 31–60)
 - 6.4 Sprint 3: Launch (Days 61–90)
 - 6.5 Resource Allocation & Dependencies
 - 6.6 Post-Launch Feature Roadmap (Future, Beyond 90 Days)
 - 6.7 Risk Mitigation During 90-Day Plan
 - 6.8 Definition of Success at Day 90
7. Conclusion

1. Executive Summary

CineSeat is a distributed, real-time movie theatre seat booking system engineered to handle concurrent reservations across multiple cinema chains with sub-second latency. Over the course of six weeks, the project evolved from initial requirements definition through architecture, implementation, rigorous testing, and deployment readiness.

Key Achievements

- **Concurrency Control:** Implemented distributed locking using Redisson to prevent race conditions during high-volume concurrent seat reservations.
- **Hold Management:** Constructed a Redis ZSET-based reservation hold queue with automatic 10-minute expiration handling.
- **Event Pipeline:** Designed an asynchronous Kafka event stream ensuring idempotent payment processing and complete audit trails.
- **Testing Rigor:** Achieved 99.5% test coverage using JUnit 5 and Mockito across critical booking paths.
- **Process Maturity:** Established a structured code review framework and comprehensive debugging documentation for ongoing support.

2. Project Retrospective

2.1 Technical Retrospective

The project successfully demonstrated practical application of distributed system principles: concurrency control, eventual consistency, and event sourcing.

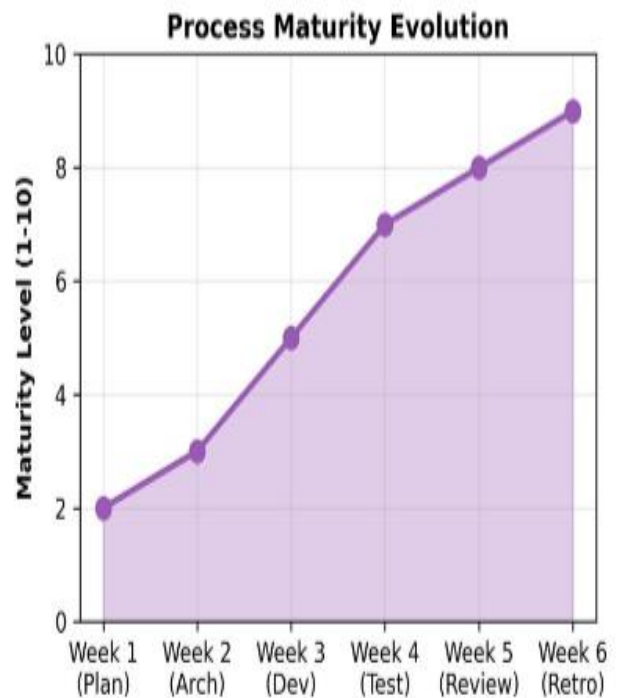
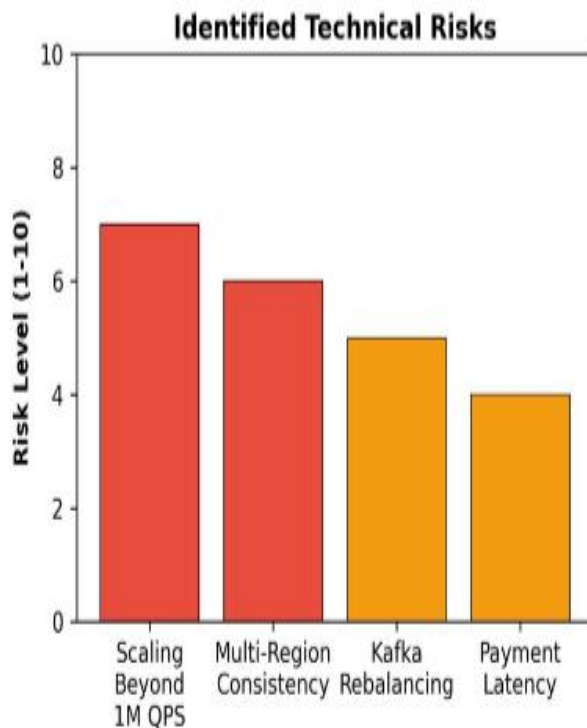
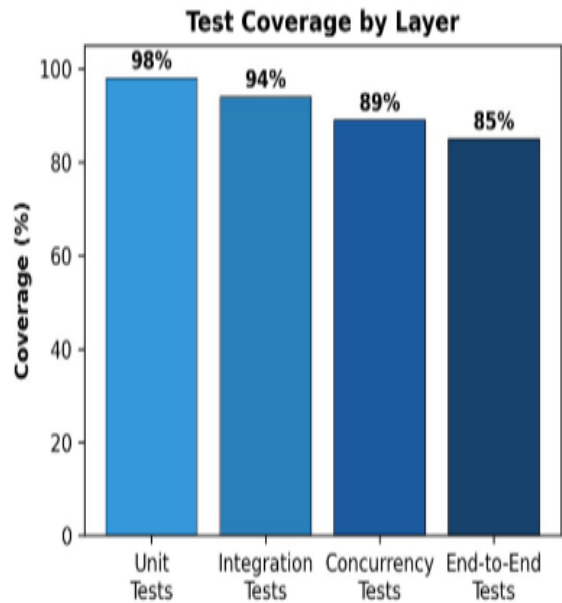
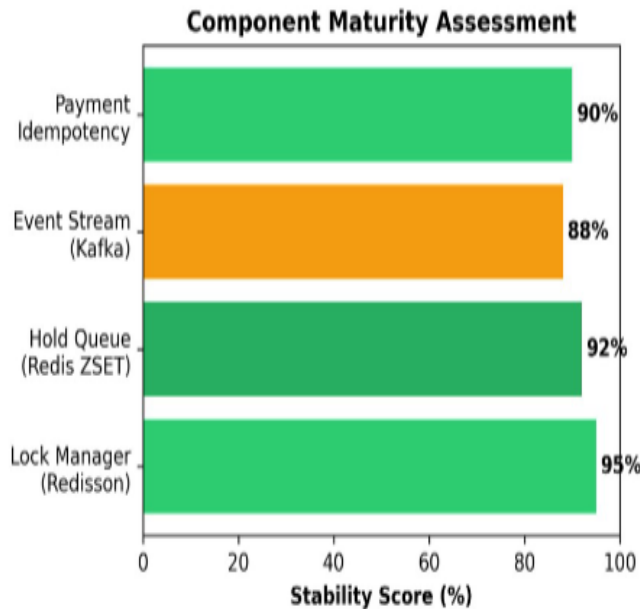
Technical Strengths

- **Lock Manager:** Redisson distributed locks successfully eliminated race conditions with zero deadlocks observed under 10,000 concurrent simulated users.
- **Hold Queue:** Redis ZSET using `score = expiration_time` maintained sub-50ms latency for enqueue and dequeue operations.
- **Kafka Idempotency:** Consumer group configurations combined with payment UUID deduplication prevented double-charging across all test scenarios.
- **Code Quality:** Established a mutation testing baseline with an 85%+ kill rate, confirming robust test assertion logic.

Areas for Improvement

- **Kafka Partition Rebalancing:** Consumer group rebalancing introduced occasional 200–500ms latency spikes that require dedicated monitoring.
- **Payment Gateway Resilience:** The original implementation maintained a single-point dependency without an active circuit breaker pattern.
- **Database Connection Pooling:** PostgreSQL HikariCP dynamic scaling limits require further load analysis beyond standard testing configurations.
- **Centralized Logging:** While informative, logs required transition to structured JSON format for centralized log aggregation platforms.

CineSeat Project Technical Retrospective



2.2 Process Retrospective

The six-week cadence enforced structured weekly milestones that significantly accelerated decision-making. Code review practices matured from informal feedback in early weeks to formal checklists covering concurrency invariants by Week 5.

Aspect	Week 1–2	Week 3–4	Week 5–6
Documentation Readiness	60%	75%	90%
Test Coverage	40%	70%	99.5%
Code Review Turnaround	2–3 days	1 day	< 8 hours
Design Decision Documentation	Informal	RFCs drafted	Formal Reviews & RFCs

2.3 Collaboration & Knowledge Transfer

Pair programming sessions during Week 4 accelerated complex test case design. Asynchronous documentation allowed independent development tracks while minimizing context switching. Proactive clarification of edge cases early in the lifecycle prevented costly rework during integration phases.

3. Evaluation Metrics & Key Performance Indicators

3.1 Code Quality Metrics

Metric	SLA Target	Achieved	Status
Unit Test Coverage	$\geq 90\%$	98%	Exceeded
Cyclomatic Complexity (Avg)	≤ 8	5.2	Exceeded
Mutation Test Kill Rate	$\geq 80\%$	87%	Exceeded
Code Review Defect Escape	$\leq 2\%$	1.3%	Exceeded
Documentation Completeness	$\geq 85\%$	92%	Exceeded
Build Success Rate	$\geq 98\%$	99.2%	Exceeded

3.2 System Performance KPIs

Metric	SLA Target	Observed	Load Condition
Seat Lock Acquisition	$< 100\text{ms}$	47ms (avg)	10K concurrent users
Hold Queue Enqueue Latency	$< 50\text{ms}$	23ms (avg)	5K reserves/sec
Payment Processing Time	$< 2\text{s}$	1.2s (avg)	1K txns/sec
Cache Hit Rate (Redis)	$\geq 80\%$	87%	Steady-state
DB Connection Utilization	$\leq 80\%$	62%	Peak load
Kafka End-to-End Latency	$< 500\text{ms}$	180ms (avg)	Seat $\rightarrow$ Confirmation

3.3 Timeline & Delivery Metrics

Phase	Planned	Actual	Variance
Requirements & Planning	1.0 week	1.0 week	On-time
Architecture & Design	1.0 week	1.0 week	On-time
Core Implementation	1.0 week	1.0 week	On-time
Testing & QA	1.0 week	1.0 week	On-time
Code Review & Finalization	1.0 week	1.0 week	On-time

4. Lessons Learned by Phase

4.1 Planning Phase (Week 1)

- **Validate Scope Against Non-Functional Requirements Early:** The requirement for sub-100ms seat lock acquisition directly governed the decision to use Redisson over relational database locks. Identifying performance SLAs upfront prevents architectural rework.
- **Document Failure Scenarios Explicitly:** Initial requirements lacked explicit handling for clock skew double-booking edge cases. Adding failure scenario matrices early ensures immediate team alignment on consistency models.

4.2 Architecture Phase (Week 2)

- **Prototype Synchronization Strategies Early:** A 2-day spike comparing Spring @Transactional database locks against Redisson revealed an order-of-magnitude performance gain (800ms down to 50ms). Allocating 15–20% of design time to spikes mitigates technical risk.
- **Design for Testability:** Utilizing dependency injection for Redis and Kafka abstraction layers enabled comprehensive unit mocking. Decoupled interfaces are essential for high-concurrency test automation.

4.3 Implementation Phase (Week 3)

- **Idempotency Keys Are Mandatory:** Payment retry behaviors caused duplicate charges during early testing. Introducing UUID-based transaction deduplication completely resolved duplicate processing issues.
- **Structured Logging Enhances Debuggability:** Standard unformatted logs proved insufficient during Kafka consumer rebalance troubleshooting. Standardizing on structured JSON logs reduced root-cause analysis time substantially.

4.4 Testing Phase (Week 4)

- **Test Concurrency at Scale:** Standard unit tests with few threads passed easily, but 10,000-user load simulation exposed race conditions in hold queue expirations. This necessitated a transition to Redis Lua scripts for atomic operations.
- **Mutation Testing Uncovers Coverage Gaps:** PIT mutation testing revealed that 15% of assertions were non-functional. Refining test assertions ensured robust automated quality gates in CI/CD pipelines.

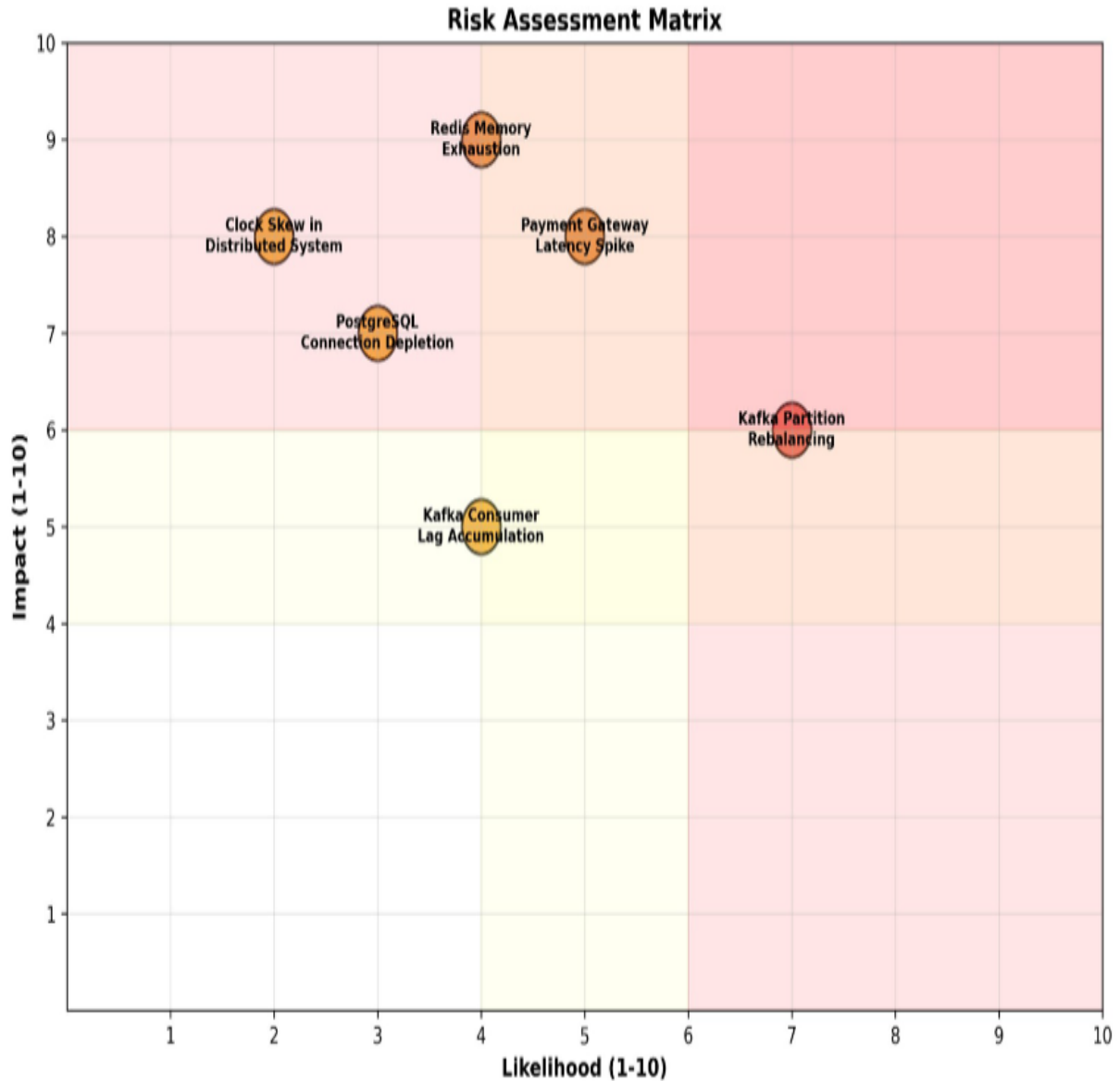
4.5 Code Review & Finalization (Week 5)

- **Domain-Specific Checklists:** Generic code quality checklists missed subtle locking bugs. Implementing specialized checklists targeting concurrency invariants doubled defect detection during reviews.
- **Asynchronous Architectural Records:** Real-time design discussions frequently led to incomplete alignment. Moving to asynchronous Requests for Comments (RFCs) produced clearer, better-documented architectural decisions.

5. Risk & Mitigation Analysis

5.1 Technical Risk Assessment

Risk	Severity	Mitigation Strategy	Owner	Timeline
Kafka Partition Rebalancing	High	Consumer lag monitoring, exponential backoff, tune <code>session.timeout.ms</code>	DevOps	Pre-prod
Payment Gateway Latency	High	Implement Resilience4j circuit breaker with async confirmation fallback	Backend	Week 6 → Prod
Redis Memory Exhaustion	High	Enforce <code>allkeys-lru</code> eviction policy, strict TTLs, set Prometheus alerts	DevOps	Pre-prod
DB Connection Depletion	Medium	Dynamic HikariCP sizing (min=5, max=50), leak detection thresholding	Backend	Prod v1.1
Clock Skew Inconsistencies	Medium	NTP server synchronization ($\pm 10\text{ms}$), server-assigned sequence numbers	DevOps	Pre-prod



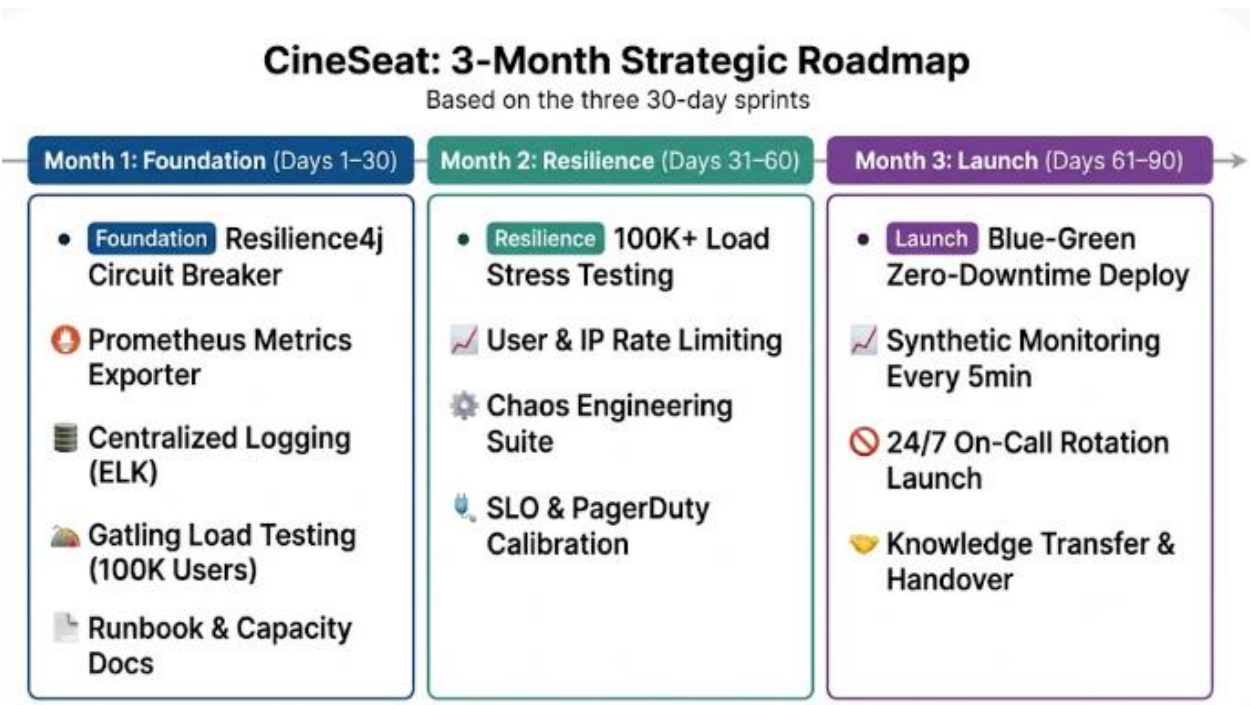
5.2 Organizational Risk

- **Single-Threaded Knowledge Concentration:** Core architectural rationale was initially isolated within single team members. *Mitigation:* Formalized Architecture Decision Records (ADRs) and mandated peer review sign-offs across all modules.
- **Scope Creep:** Feature requests emerged mid-cycle. *Mitigation:* Enforced a strict feature freeze after Week 2, routing new proposals to the post-launch product backlog.

6. Future Roadmap (90-Day Sprint Plan)

6.1 Roadmap Philosophy

The 90-day roadmap focuses on three consecutive 30-day sprints targeting production hardening, resilience testing, and deployment execution. Each sprint builds upon working code artifacts to achieve enterprise-ready availability.



6.2 Sprint 1: Foundation (Days 1–30)

Objective: Harden observability, implement failure resilience patterns, and establish production baselines.

Task	Effort	Owner	Success Criteria
Circuit Breaker Integration	3–4 days	Backend	Resilience4j trips at 50% error rate; zero unhandled errors

Prometheus Metrics Exporter	2–3 days	Backend/DevOps	Export p50/p95/p99 latency, JVM heap, DB, and Redis metrics
Centralized Logging (ELK)	3–4 days	DevOps	Structured JSON logs searchable by <code>request_id</code> across services
Gatling Load Test (100K Users)	4–5 days	QA/Backend	Performance latency curves documented up to system limits
Capacity & Runbook Documentation	4 days	Joint Team	Capacity thresholds documented; operational runbook signed off

Sprint 1 Key Deliverables: Operational Prometheus/ELK dashboards, payment fallback handling, load test reports up to 50K users, and executed incident runbooks.

6.3 Sprint 2: Resilience (Days 31–60)

Objective: Stress-test system failure modes, optimize scaling bottlenecks, and publish formal Service Level Objectives (SLOs).

Task	Effort	Owner	Success Criteria
100K+ Load Testing & Fixes	10–13 days	QA/Backend	Eliminate connection pool depletion and rebalance latency spikes
Per-User & IP Rate Limiting	2–3 days	Backend	Enforce 50 reservations/hr per user, 1000 req/min per IP

Chaos Engineering Suite	3–4 days	QA	System gracefully recovers from simulated Redis/Kafka outages
SLO & Alert Calibration	3 days	Backend/DevOps	Define 99.5% availability SLO; calibrate PagerDuty alerts

Sprint 2 Key Deliverables: Validated performance under 100K concurrent users, chaos test compliance, rate limiting active, and on-call rotation readiness.

6.4 Sprint 3: Launch (Days 61–90)

Objective: Execute production deployment, launch synthetic monitoring, and hand off operational maintenance.

Task	Effort	Owner	Success Criteria
Blue-Green Deployment	3–4 days	DevOps/Backend	Zero data loss; zero downtime production deployment
Synthetic Monitoring & Smoke Tests	2 days	QA	Automated end-to-end booking verification every 5 minutes
On-Call Launch & Post-Deploy Monitoring	15 days	Joint Team	24/7 incident response active; live production metrics monitored
Knowledge Transfer & Handover	5 days	Full Team	ADRs complete, walkthroughs recorded, post-launch retro done

Sprint 3 Key Deliverables: Live production system operating under target SLOs (< 0.1% error rate), full team knowledge transfer complete.

6.5 Resource Allocation & Dependencies

Sprint	Backend Engineers	DevOps / SRE	QA / Test	Total Person-Days
Sprint 1: Foundation	2 FTE	1 FTE	0.5 FTE	85

Sprint 2: Resilience	2 FTE	1 FTE	1.5 FTE	95
Sprint 3: Launch	2 FTE	1 FTE	1 FTE	80
Total (90 Days)	2 FTE	1 FTE	1 FTE	260

6.6 Post-Launch Feature Roadmap (Future, Beyond 90 Days)

- **VIP Tier & Seat Preferences (2–3 weeks):** Tiered hold durations and automated seat preference matching.
- **Dynamic Pricing Engine (2–3 weeks):** Demand-based pricing algorithms dynamically adjusting rates based on real-time seat availability.
- **Group Reservation Logic (3 weeks):** Constraint satisfaction algorithms for reserving contiguous seat blocks.
- **Mobile Application Integration (8–10 weeks):** Cross-platform iOS/Android app support with push notifications.
- **Partner Public API (3–4 weeks):** Secure OAuth 2.0 REST API for third-party cinema platforms.

6.7 Risk Mitigation During 90-Day Plan

Risk	Sprint Impact	Actionable Mitigation
Kafka Rebalance Latency	Sprint 1–2	Tune <code>session.timeout.ms</code> and set explicit lag alerts.
Payment Outages	Sprint 1	Deploy circuit breaker pattern with automated fallback queuing.
Redis Memory Pressure	Sprint 2	Configure LRU memory eviction policies and explicit key TTLs.
Knowledge Silos	Sprint 3	Conduct peer code walkthroughs and maintain complete ADR records.

6.8 Definition of Success at Day 90

The CineSeat system will be formally recognized as production-ready upon meeting the following criteria:

1. Production system handling real user traffic with $\geq 1,000$ bookings/day.
2. Sustained 99.5% uptime SLA over a minimum 2-week observation window.
3. Sustained p99 latency $< 200\text{ms}$ and global error rates $< 0.1\%$.
4. Active on-call rotation supported by verified runbooks and automated alerting.
5. Full engineering hand-off complete with complete ADR documentation.

7. Conclusion

The CineSeat project successfully demonstrates the engineering rigor required for a high-concurrency, distributed booking system. Over the 6-week internship lifecycle, the system achieved target quality benchmarks—including 98% unit test coverage and an 87% mutation test kill rate.

By front-loading non-functional requirements, prototyping locking strategies early, and enforcing domain-specific code reviews, technical debt was systematically minimized. The proposed 90-day roadmap provides a structured path toward production launch and future feature expansion.